



TDD ASSIGNMENT (PYBANK)



Write tests *first*, watch them fail, then write code to make them pass. 5 function tasks + 1 console app project.

semicolon _____



TASK 1 — VALIDATE_EMAIL(EMAIL)

Write a function that checks whether a string is a valid email address using three simple rules. The email must be at least 8 characters long, must contain the @ symbol and not start or end with @.

semicolon _____



TASK 2 — CALCULATE_BALANCE(TRANSACTIONS)

Write a function that accepts a list of transaction amounts (positive = deposit, negative = withdrawal) and returns the final account balance. An empty list should return 0.

semicolon _____



TASK 3 — `IS_STRONG_PASSWORD(PASSWORD)`

Write a function that returns True if a password is strong. A strong password must be at least 8 characters long.

semicolon _____



TASK 4 — APPLY_INTEREST(BALANCE, RATE, YEARS)

Write a function that computes compound interest and returns the final balance rounded to 2 decimal places.

Raise a `ValueError` if rate is negative or years is less than 1.

Formula: $\text{balance} \times (1 + \text{rate})^{\text{years}}$.

semicolon _____



TASK 5 – GET_TRANSACTION_SUMMARY(TRANSACTIONS)

Write a function that takes a list of transactions where each transaction is a two-element list ["credit", amount] or ["debit", amount] – and returns a summary as a list of lists. Each inner list holds a label and its computed value.

SAMPLE INPUT:

```
transactions = [  
    ["credit", 2000],  
    ["debit", 500],  
    ["credit", 300],  
]
```

SAMPLE OUTPUT:

```
[  
    ["total_credits", 2300],  
    ["total_debits", 500],  
    ["net_balance", 1800],  
    ["transaction_count", 3],  
]
```

semicolon





App Project – ties all five functions together into a `bank_app/` package with three files: `pybank.py` (the functions), `test_pybank.py` (integration tests), and `main.py` (the console loop). The menu covers deposits, withdrawals, balance checks, summaries, and interest application.

TDD Guide tab – the red/green/refactor cycle explained with `unittest` commands and common mistakes to avoid.

The key TDD discipline enforced throughout: tests are provided *before* any implementation, and the app project requires all tests to pass before `main.py` is written.

semicolon
